# Отношения между классами и объектами

**Александра Волосова,**
к.т.н., доцент кафедры
**ИУ5**

Презентация разработана в рамках гранта «Грант на обучение студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий. Договор № 70-2025-000850 с АНО «Аналитический центр при Правительстве Российской Федерации»

# Часть 1

## Наследование

# Наследование: is-a отношение

- Наследование: отношение "является" (is-a)
- Базовый класс (родитель) → Производный класс (потомок)
- Public наследование: интерфейс базового класса становится частью интерфейса производного
- Принцип Лисков (Liskov Substitution Principle (LSP):

- "Функции, которые используют базовый тип, должны иметь возможность использовать подтипы базового типа, **не зная об этом** "
- объекты базового класса могут быть заменены объектами производного
- LSP гарантирует, что наследование используется правильно, а иерархия классов имеет смысл и не приводит к ошибкам

# Типы наследования

public наследование:
• public члены базового → public в производном классе
• protected члены → protected в производном классе
• private члены → недоступны* в производном классе

protected наследование:
• public и protected члены → protected в производном классе
• private члены → недоступны* в производном классе

private наследование:
• public и protected члены → private в производном классе
• private члены → недоступны* в производном классе

*private члены базового класса физически присутствуют в объекте производного класса Доступ к ним можно получить через вызов **public или protected методы базового класса**, которые работают с этими private полями. Это - инкапсуляция: базовый класс скрывает свою внутреннюю реализацию, но предоставляет интерфейс для работы с ней.

# Специальные члены класса при наследовании

**Конструкторы:**

- **Не наследуются**
- **Производный класс должен вызвать конструктор базового**
- **Следует использовать списки инициализации**

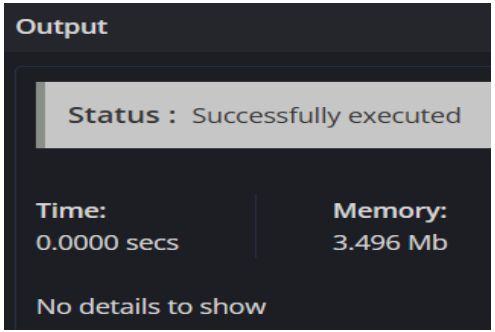
```
class Base {
public:
    Base(int x) : value(x) {}
private:
    int value;
};
class Derived : public Base {
public:
    // Ошибочный вызов. Base не имеет конструктора по умолчанию!
    Derived() {}

    // Конструктор Base вызывается через список инициализации
    Derived(int x, int y) : Base(x), extra(y) {}
 private:
    int extra;
};
```

# Производному классу не передаются:

**Конструкторы** (кроме C++11 и новее с using, но это не является наследованием в классическом смысле)

```
#include <bits/stdc++.h>
using namespace std;
class Base {
public:    Base(int x) {}
};
class Derived : public Base {
public:    using Base::Base; // Используются конструкторы Base};
int main() {
Derived d(5);
return 0;}
```

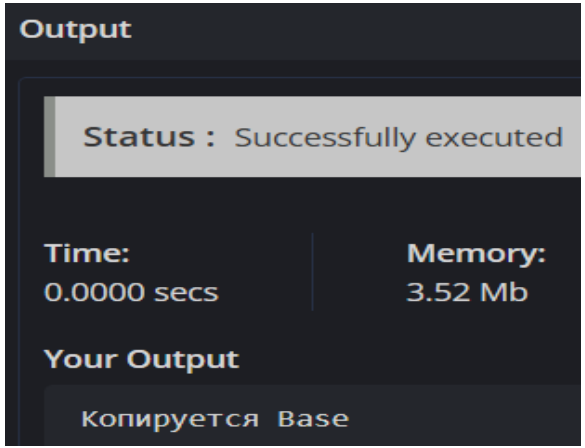


# Конструктор копирования

```
class Base {
public:
    Base() = default;
    Base(const Base& other) {
        cout << "Копируется Base\n";
    }
};

class Derived : public Base {
    // Нет своих конструкторов копирования
};

int main() {
    Derived d1;
    Derived d2(d1); // Будет вызван Base::Base(const Base&)
    return 0;
}
```




Компилятор автоматически генерирует для Derived **конструктор копирования**, который:

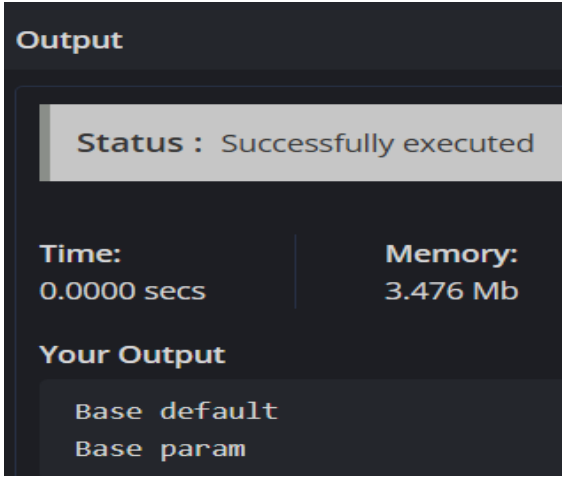
- вызывает конструктор копирования базового класса
- копирует члены Derived (при наличии)

# Конструктор по умолчанию

```
class Base {
public:
    Base() { cout << "Base default\n"; }
    Base(int x) { cout << "Base param\n"; }
};

class Derived : public Base {
public:
};

int main() {
    Derived d1;     // Вызовет Base()
    Derived d1(5);
return 0;
}
```




Если у производного класса нет своих конструкторов, компилятор автоматически генерирует **конструктор по умолчанию**, который вызывает конструктор по умолчанию базового класса.

# Специальные члены класса при наследовании

**Копирующие операции:**

- **Не наследуются автоматически**
- **Должны явно вызывать базовые версии**

```
class Base {
public:
   Base() = default;
   Base(const Base& other) : data(other.data) {}
   Base& operator=(const Base& other) {
      if (this != &other) data = other.data;
      return *this;
   }
private:
   int data;
};
class Derived : public Base {
public:
   // Конструктор копирования
   Derived(const Derived& other)
// Вызывается копирующий конструктор Base
      : Base(other), extra(other.extra) {}
```

```
// Оператор присваивания
   Derived& operator=(const Derived& other)
{
      if (this != &other) {
// Явно вызывается присваивание Base
         Base::operator=(other);
         extra = other.extra;
      }
      return *this;
   }

private:
   int extra;
};
```

# Специальные члены класса при наследовании

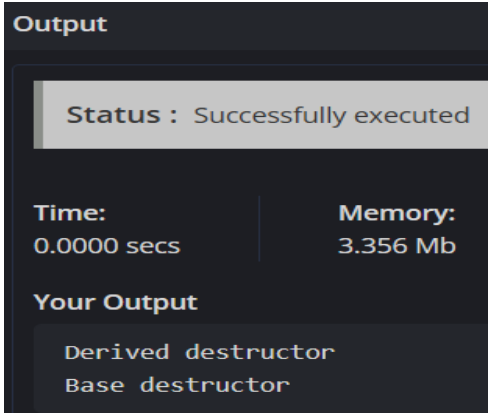
## Деструкторы:

- Вызываются в обратном порядке (производный → базовый)
- Должны быть виртуальными для полиморфных классов

```
class Base {
public:
    virtual ~Base() { cout << "Base destructor\n"; }
};

class Derived : public Base {
public:
    ~Derived() { cout << "Derived destructor\n"; }
};

int main() {
    Base* ptr = new Derived();
    delete ptr;  // Выведет: Derived destructor → Base destructor
}
```



```
class Base { public: int a; };
class Derived : public Base { public: int b; };

int main() {
   Derived d;
   Base* ptr = &d;       // Указатель на Base смотрит на Derived
   ptr->a = 5;
```

// **возникает ошибка компиляции** - попытка доступа к члену производного класса через указатель на базовый класс

```
   ptr->b = 10 ;}
```

**Error**

```
sol.cpp: In function 'int main()':
sol.cpp:10:10: error: 'class Base' has no member named 'b'
   10 |     ptr->b = 10;
      |          ^
```

- В памяти объекта d есть **оба поля**: a (унаследовано) и b (собственное)
- Указатель ptr указывает на начало части Base
- **Тип указателя определяет видимую часть объекта**
- Для объекта Derived, через указатель Base* доступны **только члены Base**
- Это - **статическая типизация**: компилятор ориентируется на тип указателя, а не на реальный объект. Если бы на этапе компиляции было бы разрешено ptr->b, то при вызове с обычным Base произошла бы **запись в чужую память. Работа через базовый указатель ограничена интерфейсом базового класса**
- Присваивание Base* ptr = &d возможно, потому что Derived **is-a** Base

## Производному классу не передаются:

- **Деструктор** (но вызывается при уничтожении наследника)
- **Оператор присваивания**
- **Дружественные отношения** (friend). Если класс A объявил другом функцию f(), а класс B наследуется от класса A, функция f() не становится автоматически другом класса B

# Пример иерархии наследования

```
// Базовый класс
class Shape {
public:
    virtual void draw() const = 0;  // Чисто виртуальная функция
    virtual double area() const = 0;
    virtual ~Shape() {}             // Виртуальный деструктор
protected:
    Point center;
};
// Public наследование
class Circle : public Shape {
public:
    Circle(Point c, double r) : center(c), radius(r) {}

    void draw() const override {
        // Реализация рисования круга
    }

    double area() const override {
        return 3.14159 * radius * radius;
    }
private:
    double radius;
};
```

```
// Еще один производный класс
class Rectangle : public Shape {
    // ...
};
```

## Пример конструкторов

```
class Base {
public:
    Base(int x) : value(x) {}
    Base(const Base& other) : value(other.value) {}
    virtual ~Base() {}

protected:
    int value;
};

class Derived : public Base {
public:
    // Должен вызвать конструктор Base
    Derived(int x, std::string n)
        : Base(x), name(n) {}  // Правильно: в списке инициализации

    // Derived(int x, std::string n) {
    //     Base(x);            // НЕПРАВИЛЬНО! Это создает временный объект
    //     name = n;
    // }

    // Конструктор копирования
    Derived(const Derived& other)
        : Base(other),          // Вызываем конструктор копирования Base
          name(other.name) {}   // Копируем члены Derived

private:
    std::string name;
};
```

# Области видимости в иерархиях

- Каждый класс имеет свою область видимости
- Поиск имени начинается в текущем классе
- Затем поднимается по иерархии наследования
- Сокрытие имен: функция в производном классе скрывает функции с тем же именем в базовом

# Пример сокрытия имен

```
class Base {
public:
    virtual void f(int x) { std::cout << "Base::f(int)" << std::endl; }
    void g(double d) { std::cout << "Base::g(double)" << std::endl; }
};

class Derived : public Base {
public:
    // Скрывает Base::f(int), даже с другим типом параметра
    void f(double x) { std::cout << "Derived::f(double)" << std::endl; }

    // Правильно: override указывает, что переопределяем
    void f(int x) override { std::cout << "Derived::f(int)" << std::endl; }

    // Скрывает Base::g(double)
    void g(int i) { std::cout << "Derived::g(int)" << std::endl; }
};

// Использование
Derived d;
d.f(5);       // Вызывает Derived::f(int)
d.f(5.0);     // Вызывает Derived::f(double)
d.g(5);       // Вызывает Derived::g(int)
d.g(5.0);     // Вызывает Derived::g(int) - неявное преобразование
// d.Base::g(5.0);  // Явный вызов скрытой функции
```

# Часть 2

## Множественное наследование

# Множественное наследование

- Класс может наследовать от нескольких базовых классов
- Проблемы: конфликты имен, неоднозначность
- Решения: явное разрешение области видимости, виртуальное наследование

# Пример множественного наследования

```
// Интерфейсы
class Drawable {
public:
    virtual void draw() const = 0;
    virtual ~Drawable() {}
};
class Clickable {
public:
    virtual void onClick() = 0;
    virtual ~Clickable() {}
};
// Класс может реализовать несколько интерфейсов
class Button : public Drawable, public Clickable {
public:
    void draw() const override {
        // Реализация рисования
    }
    void onClick() override {
        // Обработка клика
    }
    // Конфликт имен, если оба базовых класса имеют функцию с одинаковым именем
    // Решение: явное указание
    void commonFunction() {
        Drawable::commonFunction();  // Вызываем из Drawable
        Clickable::commonFunction(); // Вызываем из Clickable
    }
};
```

# Виртуальное наследование

- Решает проблему дублирования базового класса при множественном наследовании
- virtual в объявлении наследования
- Виртуальный базовый класс создается только один раз
- Конструктор виртуального базового класса вызывается самим производным классом
- Страуструп: «Избегайте виртуального наследования, если возможно»
- Липпман: «Виртуальное наследование дорого с точки зрения производительности»

# Проблема ромба

Проблема ромба:

A (базовый класс)

↙ ↘

B  C (наследуют от A)

↘ ↙

D (наследует от B и C)

**Без виртуального наследования:**

- D содержит два экземпляра A
- Неоднозначность при доступе к членам A

# Пример проблемы ромба

```
class A {
public:
    int value;
    A(int v) : value(v) {}
};

// Обычное наследование (проблема)
class B : public A {
public:
    B() : A(1) {}
};

class C : public A {
public:
    C() : A(2) {}
};

class D : public B, public C {
    // Содержит два экземпляра A!
    // Один от B со значением 1
    // Второй от C со значением 2
};
```

```
// Виртуальное наследование (решение)
class B2 : virtual public A {
public:
    B2() : A(1) {}
};

class C2 : virtual public A {
public:
    C2() : A(2) {}
};

class D2 : public B2, public C2 {
public:
    D2() : A(3) { // Вызов конструктора A
//Содержит один экземпляр A со значением 3
    }
};
```

# Практические рекомендации

**Страуструп:**

1. Предпочитайте композицию наследованию
2. Используйте public наследование только для is-a отношений
3. Избегайте множественного наследования от классов с реализацией
4. Делайте деструкторы виртуальными в полиморфных классах
5. Используйте override для ясности

**Липпман:**

1. Придерживайтесь правила LSP (Liskov Substitution)
2. Используйте using для решения конфликтов имен
3. Избегайте виртуального наследования, если возможно
4. Всегда вызывайте конструкторы базовых классов в списке инициализации
5. Используйте множественное наследование только для интерфейсов

# Директива using

- директива using  делает члены базового класса доступными в производном
- Решает проблему сокрытия имен
- Можно использовать для конструкторов (C++11)
- Страуструп: «using — способ управления областью видимости»
- Липпман: «Используйте using для наследования конструкторов»

# Пример директивы using

```
class Base {
public:
    void f(int) {}
    void f(double) {}
    void g() {}
    Base(int x) {}
};

class Derived : public Base {
public:
    // Делаем все версии Base::f доступными
    using Base::f;

    // Наследуем конструкторы (C++11)
    using Base::Base;

    // Добавляем свою версию
    void f(const std::string&) {}

    // Без using Base::g, эта функция скрыла бы Base::g
    void g(int) {}
};
```

```
// Использование
// Использует унаследованный конструктор
Derived d(42);
d.f(5);
Base::f(int)
d.f(3.14);
Base::f(double)
d.f("hello");
Derived::f(const std::string&)
// Ошибка: Base::g скрыта
d.g();
// Явный вызов
// d.Base::g();
```

# Выводы

- Предпочтительнее использовать композицию и агрегацию: has-a отношения
- Для реализации полиморфизма используется наследование: is-a отношения
- Public-наследование удобно для интерфейсов, private - для реализации
- Следует учитывать особенности работы конструкторов и деструкторов в иерархиях
- Следует использовать override и using для избежания ошибок
- Множественное наследование сложно, но полезно для интерфейсов
- Виртуальное наследование решает проблему ромба, но дорого обходится
- Всегда соблюдайте принцип подстановки Лисков (LSP)

# Часть 3

## Наследование в C++17/20/23)

# Эволюция наследования в современном C++

- C++11/14 заложили основу через final, override, делегирующие конструкторы

- C++17/20/23 предоставили новые инструменты для проектирования и заменяя некоторые традиционные сценарии использования наследования

# Ключевое слово final (C++11)

**Для классов:** Запрещает дальнейшее наследование от класса.

class Base final { /* ... */ };

*class Derived : public Base {}; // Ошибка компиляции*

**Для виртуальных функций:** Запрещает переопределение метода в производных классах.

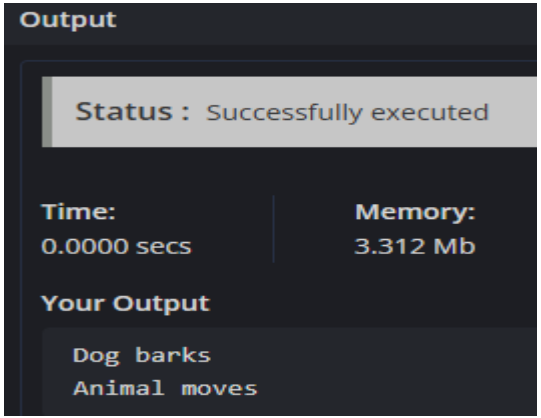
class Base { virtual void foo() final;

*// Никто не может переопределить foo* };

```cpp
#include <iostream>
// Пример 1: final для класса
class Base final {
public:
   void show() { std::cout << "Base::show()" << std::endl; }
};
// Ошибка компиляции! Нельзя наследовать от final класса
// class Derived : public Base { };
// Пример 2: final для виртуальной функции
class Animal {
public:
   virtual void sound() const {
      std::cout << "Animal makes sound" << std::endl;
   }

   virtual void move() const final {
// Нельзя переопределить
      std::cout << "Animal moves" << std::endl;
   }
};
class Dog : public Animal {
public:
   void sound() const override {  // OK
      std::cout << "Dog barks" << std::endl;
   }
   // Ошибка! Нельзя переопределить final функцию
   // void move() const override { }
};
```

```cpp
int main() {
    Dog d;
    d.sound();  // Dog barks
    d.move();   // Animal moves
    return 0;
}
```



# Наследование конструкторов: от C++11 к C++17

**Проблема:** Конструкторы базового класса не наследуются по умолчанию.

- **Решение C++11:**

  using Base::Base;. Это позволяло использовать конструкторы в производном классе, но могло привести к проблемам, если в производном классе есть свои члены.

- **Решение C++17:** Если в производном классе объявлены **пользовательские конструкторы**, то наследуемые конструкторы базового класса **не генерируются** автоматически. Это сделано для повышения ясности и безопасности кода.

```cpp
#include <iostream>
#include <string>
class Base {
public:
   Base() : value(0) {
      std::cout << "Base default constructor" << std::endl;
   }
   Base(int v) : value(v) {
      std::cout << "Base(int) constructor: " << v << std::endl;
   }
   Base(const std::string& s) : value(s.length()) {
      std::cout << "Base(string) constructor: " << s << std::endl;
   }
   int value;
};
// C++11: наследование конструкторов через using
class Derived11 : public Base {
public:
   using Base::Base;  // Наследуем все конструкторы Base
// Добавляем свой конструктор
   Derived11(int v, double d) : Base(v), extra(d) {
      std::cout << "Derived11(int, double) constructor" << std::endl;
   }
   double extra = 0.0;
};
// C++17: если есть пользовательский конструктор,
//то наследуемые конструкторы не генерируются автоматически
class Derived17 : public Base {
public:
   using Base::Base;  // Явно наследуем конструкторы Base

   // Свой конструктор
   Derived17(bool flag) : Base(flag ? 1 : 0), is_flag(flag) {
      std::cout << "Derived17(bool) constructor" << std::endl;
   }
   bool is_flag = false;
};
```

```cpp
int main() {
   // C++11
   Derived11 d1(42);          // OK: Base(int)
   Derived11 d2("hello");       // OK: Base(string)
   Derived11 d3(100, 3.14);     // OK: Derived11(int, double)

   // C++17
   Derived17 d4(true);          // OK: Derived17(bool)
   Derived17 d5(99);            // OK: Base(int) все еще доступен
   // Derived17 d6("world");     // OK: Base(string) тоже доступен

   return 0;
}
```

**Output**

**Status :** Successfully executed

| Time: | Memory: |
|---|---|
| 0.0000 secs | 3.536 Mb |

**Your Output**

```
Base(int) constructor: 42
Base(string) constructor: hello
Base(int) constructor: 100
Derived11(int, double) constructor
Base(int) constructor: 1
Derived17(bool) constructor
Base(int) constructor: 99
```

# Наследование конструкторов: от C++11 к C++17

В C++17 классы, имеющие открытые базовые классы, могут быть **агрегатами:**

- Это позволяет инициализировать базовый класс напрямую

```
struct Base { int x; };
struct Derived : Base { int y; };
Derived d{{1}, 2};
// C++17: инициализация
Base.x = 1, Derived.y = 2
```

- Упрощает код, избавляя от необходимости писать конструкторы для простых структур-оберток

# Агрегат

— это особый тип класса (или структуры), который можно инициализировать напрямую, перечисляя значения его членов в фигурных скобках {}

Агрегат:

- **Не имеет виртуальных функций**
- **Не скрывает свои данные**
- **Не имеет пользовательских конструкторов**

```
struct Point {
    int x;
    int y;
};

struct Person {
    std::string name;
    int age;
    double salary;
};

struct Mixed : Base {
int data;
    std::string text;
};
```

**C++11/14**

- Агрегат — это класс без:
- Пользовательских конструкторов
- Приватных/защищенных нестатических членов
- Базовых классов
- Виртуальных функций

**C++17**

Агрегат — это класс без:

- Пользовательских конструкторов
- Приватных/защищенных нестатических членов
- Виртуальных функций

Может иметь базовые классы, но они тоже должны быть агрегатами

Может иметь **= default** или **= delete** конструкторы (нея вляются пользовательскими в **C++17)**

**C++20**

Агрегат — это класс без:

- Приватных/защищенных нестатических членов
- Виртуальных функций
- Пользовательских конструкторов

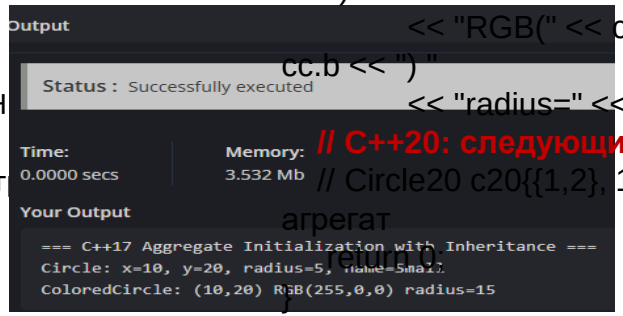
= default лишает агрегат его статуса

Может иметь базовые классы (они тоже должны быть агрегатами)

Пользовательский конструктор уничтожает статус агрегата

```
#include <iostream>
#include <string>
// Базовые структуры
struct Point {
    int x = 0;
    int y = 0;
};
struct Color {
    int r = 0;
    int g = 0;
    int b = 0;
};
// C++17: наследник может быть агрегатом!
struct Circle : Point {
    int radius = 1;
    std::string name;
};
// Пример с множественным наследованием
struct ColoredCircle : Point, Color {
    int radius = 5;
};
// C++20: класс с = default конструктором Н
struct Circle20 : Point {
    Circle20() = default;  // Делает класс НЕ аг
    int radius = 1;
};
```

```
int main() {
    std::cout << "===
C++17 Aggregate Initialization with Inheritance ==="
              << std::endl;
    // Прямая агрегатная инициализация
базового класса!
    Circle c1{{10, 20}, 5, "Small"};
    // Point{x=10, y=20}, radius=5, name="Small"
    std::cout << "Circle: x=" << c1.x << ", y=" << c1.y
              << ", radius=" << c1.radius << ", name=" <<
c1.name
              << std::endl;
    // Можно опускать скобки для базового класса
    Circle c2{{}, 3, "Default"};  // Point{0,0}
   // Множественное наследование
    ColoredCircle cc{{10, 20}, {255, 0, 0}, 15};
    // Point{10,20}, Color{255,0,0}, radius=15
    std::cout << "ColoredCircle: (" << cc.x << "," << cc.y
<< ") "
              << "RGB(" << cc.r << "," << cc.g << "," <<
cc.b << ") "
              << "radius=" << cc.radius << std::endl;
    // C++20: следующий код работать НЕ будет
    // Circle20 c20{{1,2}, 10};  // Ошибка! Circle20 не
агрегат
    std::cout << "
}
```



# Изменения в агрегатах (C++20)

- В C++20 класс с любым объявленным пользователем конструктором (включая = default или = delete) **не является агрегатом**

- Это важно учитывать при проектировании иерархий, чтобы понимать, как можно инициализировать объекты.

```
#include <iostream>
#include <type_traits>
struct Base { int b = 0; };
// Демонстрация классов
struct WithDefaultCtor : Base {
   WithDefaultCtor() = default;  // Ключевой момент!
   int d = 0;
};
struct PureAggregate : Base {
   int d = 0;  // Нет конструкторов
};
int main() {
   std::cout << std::boolalpha;
   std::cout << "C++ standard: " << __cplusplus << std::endl;
// Проверка статуса агрегата
  std::cout << "\n=== Статус агрегатов ===\n";
  std::cout << "WithDefaultCtor is aggregate? "
          << std::is_aggregate_v<WithDefaultCtor> << std::endl;
  std::cout << "PureAggregate is aggregate? "
          << std::is_aggregate_v<PureAggregate> << std::endl;
   // Демонстрация инициализации
   std::cout << "\n=== Инициализация ===\n";

   // PureAggregate всегда можно инициализировать как агрегат
   PureAggregate p{{1}, 2};
   std::cout << "PureAggregate: b=" << p.b << ", d=" << p.d <<
std::endl;

  // WithDefaultCtor - зависит от стандарта
   #if __cplusplus >= 202002L
      // C++20 и новее - не агрегат
      WithDefaultCtor w;  // только конструктор по умолчанию
      std::cout << "C++20+: используем конструктор по
умолчанию\n";
   #else
      // C++17 и старше - еще агрегат
      WithDefaultCtor w{{1}, 2};
      std::cout << "C++17: агрегатная инициализация работает\n";
   #endif

   std::cout << "WithDefaultCtor: b=" << w.b << ", d=" << w.d <<
std::endl;
    return 0; }
```

Output

Status : Successfully executed

| Time: | Memory: |
| --- | --- |
| 0.0000 secs | 3.48 Mb |

Your Output

```
C++ standard: 202002

=== Статус агрегатов ===
WithDefaultCtor is aggregate? false
PureAggregate is aggregate? true

=== Инициализация ===
PureAggregate: b=1, d=2
C++20+: используем конструктор по умолчанию
WithDefaultCtor: b=0, d=0
```